

ORQUESTACIÓN DE CONTENEDORES. KUBERNETES

CONFIGURACIÓN Y OPTIMIZACIÓN DE SISTEMAS DE CÓMPUTO
ENRIQUE SOPEÑA URBANO

Índice

INTRODUCCIÓN	2
¿Qué es la Orquestación de Contenedores?	2
¿Por qué Kubernetes es el Estándar Actual?	2
FUNDAMENTOS DE KUBERNETES	3
Linux cgroups y namespaces	3
ARQUITECTURA DEL CLÚSTER KUBERNETES	4
Componentes del Control Plane	4
Componentes de los Worker Nodes	4
OBJETOS Y RECURSOS FUNDAMENTALES DE KUBERNETES	6
Pods	6
Deployments	6
Services	6
ReplicaSet / Replication Controller	7
Labels y Selectors	7
REDES EN KUBERNETES	9
CNI: Calico, Flannel y Weave	9
Acceso Externo: NodePort, LoadBalancer e Ingress	9
HPA – HORIZONTAL POD AUTOSCALER	10
ALMACENAMIENTO EN KUBERNETES	11
Persistent Volumes (PV)	11
Persistent Volume Claims (PVC)	11
CONCLUSIONES	12
BIBLIOGRAFÍA	13

INTRODUCCIÓN

El panorama de la informática distribuida y el desarrollo de aplicaciones modernas ha experimentado una transformación significativa con la adopción generalizada de la arquitectura de contenedores. Esta práctica permite encapsular aplicaciones y sus dependencias, asegurando la portabilidad y la consistencia del entorno de ejecución a través de diversos hosts. No obstante, gestionar grandes volúmenes de contenedores en entornos de producción complejos, escalar las cargas de trabajo de manera eficiente y mantener la alta disponibilidad exige herramientas sofisticadas. Es en este contexto donde la orquestación de contenedores emerge como una disciplina esencial para la gestión automatizada de las aplicaciones.

¿Qué es la Orquestación de Contenedores?

La orquestación de contenedores se define como el conjunto de procesos automatizados que aprovisionan, despliegan, escalan y gestionan las aplicaciones contenidas en contenedores, sin la necesidad de intervención manual o de preocuparse por la infraestructura subyacente. La meta fundamental es garantizar que los contenedores se ejecuten de manera óptima, independientemente del host físico o virtual en el que residan.

Las funciones principales de la orquestación abarcan:

- **Aprovisionamiento de recursos:** Asignación automática de los recursos de cómputo, red y almacenamiento necesarios para la ejecución del contenedor.
- **Despliegue y Scheduling:** Distribución inteligente de los contenedores a través del clúster basándose en políticas de recursos y restricciones definidas.
- **Escalado y Balanceo de Carga:** Capacidad de aumentar o reducir el número de instancias de contenedores para responder a la demanda de tráfico y distribuir las peticiones entrantes.
- **Gestión del Ciclo de Vida:** Incluye la monitorización del estado, la auto-recuperación ante fallos del host o del contenedor, y la gestión de actualizaciones (rolling updates).

¿Por qué Kubernetes es el Estándar Actual?

Kubernetes (k8s) se ha establecido como la plataforma dominante para la orquestación a nivel enterprise desde que Google liberó el proyecto en el año 2014. Si bien existían soluciones previas como Docker Compose (orientado a un único host físico o virtual), Docker Swarm o Apache Mesos, Kubernetes se distingue por su enfoque en la gestión de clústeres a gran escala.

La característica distintiva de k8s es su modelo de Administración Declarativa de Objetos. En lugar de emitir comandos imperativos (hacer esto, luego aquello), los usuarios definen el estado deseado del sistema (por ejemplo, "quiero 3 réplicas de la aplicación web X") utilizando archivos de configuración YAML. El Control Plane de Kubernetes se encarga continuamente de comparar el estado actual del clúster con el estado declarado y de ejecutar las acciones correctivas necesarias para alcanzar la convergencia.

FUNDAMENTOS DE KUBERNETES

La plataforma Kubernetes funciona como un sistema operativo para el clúster. Su diseño se enfoca en proporcionar una capa de abstracción que convierte un conjunto de máquinas individuales en un único recurso de cómputo unificado, administrado a través de su plano de control (Control Plane). Este sistema se fundamenta en la Administración Declarativa, donde el usuario especifica el estado final deseado de la infraestructura, y Kubernetes se encarga del Estado Actual del clúster para que converja hacia el Estado Deseado. Esta filosofía reduce la complejidad operativa al delegar las tareas de gestión y auto-recuperación al orquestador. El núcleo de su funcionamiento se sustenta en la creación y manipulación de Objetos (como Pods, Services, Deployments), que son las representaciones persistentes del estado de las cargas de trabajo y que se almacenan de manera segura en la base de datos distribuida del clúster, etcd.

El funcionamiento de Kubernetes no se basa en una tecnología de virtualización hipervisora tradicional, sino que se construye fundamentalmente sobre dos características intrínsecas del núcleo de Linux: los namespaces y los cgroups (Control Groups). Estos mecanismos, desarrollados nativamente, son los que proporcionan el aislamiento y la gestión de recursos necesarios para definir y ejecutar un contenedor.

Linux cgroups y namespaces

Los namespaces tienen la función de actuar como la capa de aislamiento de los procesos. Su misión es garantizar que un contenedor perciba un entorno de sistema operativo propio y separado del host subyacente y de otros contenedores en ejecución. Este aislamiento se consigue al virtualizar y particionar recursos críticos como los identificadores de proceso (PID), la pila de red (NET), los puntos de montaje del sistema de archivos (MNT), y los identificadores de usuario y grupo (USER). Gracias a esta división, un proceso dentro de un contenedor opera con la ilusión de tener su propio entorno de sistema operativo dedicado.

Complementando el aislamiento proporcionado por los namespaces, los cgroups son el mecanismo responsable de la limitación y contabilidad del uso de recursos de hardware. Su implementación es esencial para la estabilidad del clúster, ya que los cgroups imponen límites estrictos sobre la cantidad de recursos que cualquier grupo de procesos —es decir, cualquier contenedor— puede consumir. Esto incluye la limitación del tiempo de la Unidad Central de Procesamiento (CPU), la asignación de memoria (RAM), la priorización de la entrada/salida de disco (I/O) y el control del ancho de banda de red. En síntesis, los namespaces proporcionan la separación lógica, y los cgroups la gobernanza de los recursos físicos para prevenir el agotamiento de los nodos y asegurar una ejecución justa para todas las cargas de trabajo.

ARQUITECTURA DEL CLÚSTER KUBERNETES

La arquitectura de Kubernetes opera bajo un modelo distribuido que se divide conceptualmente en dos conjuntos de nodos con funciones diferenciadas: el Plano de Control (Control Plane) y los Nodos Trabajadores (Worker Nodes). Esta separación es fundamental para garantizar la estabilidad, la escalabilidad y la tolerancia a fallos del sistema, ya que las responsabilidades de gestión se mantienen aisladas de las cargas de trabajo de las aplicaciones. El Control Plane se encarga de mantener el estado deseado del cluster, mientras que los Worker Nodes ejecutan las tareas de cómputo, red y almacenamiento definidas por dicho estado.

Componentes del Control Plane

El Control Plane se puede concebir como el cerebro del cluster. Estos componentes suelen ejecutarse en uno o varios hosts dedicados (los antiguos Master Nodes) para proporcionar alta disponibilidad. Su propósito es tomar decisiones globales sobre el cluster y responder a eventos específicos, como el inicio o la detención de un Pod.

La base de datos distribuida etcd es el repositorio central de clave-valor que almacena el estado completo y persistente de todo el cluster, incluyendo las especificaciones del estado deseado de la aplicación. Su fiabilidad y consistencia son críticas, por lo que su configuración en alta disponibilidad es una práctica estándar.

El API Server es el punto de entrada principal para la comunicación dentro del cluster y es el único componente que interactúa directamente con etcd. Este componente expone la API REST de Kubernetes y se encarga de validar y procesar las operaciones de creación, actualización y eliminación de los objetos, sirviendo como front-end unificado.

El Scheduler es responsable de asignar un Pod recién creado a un Worker Node específico. Para ello, examina los requisitos de recursos del Pod (CPU, memoria) y las condiciones actuales de los nodos (capacidad disponible, afinidad o antiafinidad) para tomar la decisión de emplazamiento más adecuada.

Finalmente, el Controller Manager es un proceso que agrupa lógicamente una serie de controladores (p. ej., el Node Controller, el Replication Controller o el Endpoint Controller). Estos controladores monitorizan el estado real del cluster a través del API Server y lo comparan con el estado deseado; si se detecta una divergencia, el controlador toma medidas correctivas para restablecer la convergencia.

Componentes de los Worker Nodes

Los Nodos Trabajadores, o simplemente Nodos, son las máquinas que proporcionan la potencia de cómputo para ejecutar las cargas de trabajo (Pods). Cada Worker Node contiene tres componentes fundamentales que permiten su participación activa en el cluster.

El kubelet es el agente principal que se ejecuta en cada Worker Node y es el motor de comunicación bidireccional entre el Control Plane y el nodo. Este componente recibe especificaciones de Pods del API Server y se asegura de que los contenedores definidos en ese

Pod se estén ejecutando y se encuentren en buen estado. El kubelet también reporta el estado de salud del nodo y sus recursos al Control Plane.

El kube-proxy es un agente de red que mantiene las reglas de red en el nodo. Su función es implementar la abstracción de Service de Kubernetes, gestionando el tráfico de red de forma eficiente. El kube-proxy se encarga de la comunicación de red dentro y fuera de los Pods, facilitando el enrutamiento y el balanceo de carga entre las instancias de las aplicaciones que conforman un Service.

Por último, el Container Runtime (p. ej., containerd o CRI-O) es el software responsable de ejecutar los contenedores. Este componente interactúa con los mecanismos de Linux (cgroups y namespaces) para descargar, desempaquetar y ejecutar la imagen de contenedor, asegurando el aislamiento adecuado de la carga de trabajo.

OBJETOS Y RECURSOS FUNDAMENTALES DE KUBERNETES

La arquitectura de Kubernetes se sustenta en un modelo de objetos persistentes que representan el estado deseado del clúster. Estos objetos son registros de intención que el usuario crea, usualmente mediante archivos YAML, y que el sistema se encarga de materializar y mantener. A continuación, se ofrece un análisis exhaustivo de las abstracciones primarias.

Pods

El Pod constituye la unidad atómica de programación y ejecución en Kubernetes. A diferencia de un contenedor individual, un Pod encapsula uno o varios contenedores fuertemente acoplados que comparten un ciclo de vida, almacenamiento y recursos de red. Conceptualmente, un Pod modela un "host lógico" específico para una aplicación; en un entorno pre-contenedorizado, esto equivaldría a ejecutar múltiples procesos en la misma máquina física o virtual.

Los contenedores dentro de un mismo Pod comparten la misma pila de red (IP y puertos) y el mismo espacio de nombres IPC (Comunicación entre Procesos), lo que les permite comunicarse entre sí a través de localhost con una latencia mínima. Además, tienen acceso a volúmenes de almacenamiento compartidos, lo que facilita el intercambio de datos efímero o persistente. Es importante destacar que los Pods son diseñados como entidades efímeras y desechables; no se "reparan" ante un fallo, sino que se reemplazan por nuevas instancias con una identidad distinta, lo que obliga a gestionar su direccionamiento a través de abstracciones de nivel superior.

Deployments

El objeto Deployment actúa como una capa de abstracción de alto nivel que gestiona la creación y el escalado de Pods y ReplicaSets. Su función principal es permitir la administración declarativa de las actualizaciones de las aplicaciones. El usuario describe el estado deseado en el objeto Deployment (por ejemplo, cambiar la versión de la imagen del contenedor), y el controlador del Deployment cambia el estado actual al estado deseado a una velocidad controlada.

Este recurso es fundamental para gestionar el ciclo de vida de aplicaciones sin estado (stateless). Entre sus capacidades avanzadas se encuentra la ejecución de Rolling Updates (actualizaciones continuas), que permiten actualizar las aplicaciones con tiempo de inactividad cero reemplazando progresivamente las instancias antiguas por las nuevas. Asimismo, los Deployments mantienen un historial de revisiones, lo que faculta al administrador para realizar un Rollback (reversión) a una versión anterior estable en caso de que la nueva despliegue errores críticos.

Services

Dada la naturaleza volátil de los Pods, cuyas direcciones IP cambian cada vez que son recreados, Kubernetes introduce el concepto de Service para garantizar la conectividad fiable. Un Service es una abstracción que define un conjunto lógico de Pods y una política de acceso a ellos, proporcionando una dirección IP virtual estable (ClusterIP) y un nombre DNS inmutable.

El Service desacopla el acceso a la aplicación de la infraestructura subyacente. Cuando un cliente envía una solicitud a la IP del servicio, este actúa como un balanceador de carga interno, distribuyendo el tráfico de red entre los Pods disponibles que coinciden con su selector. Esto asegura que la comunicación entre microservicios o desde el exterior sea fluida, independientemente de la rotación, el escalado o la reprogramación de los Pods en diferentes nodos del clúster.

ReplicaSet / Replication Controller

El Replication Controller es la primitiva original diseñada para asegurar la disponibilidad de un número específico de réplicas de un Pod. Sin embargo, ha sido sucedido casi en su totalidad por el ReplicaSet. Aunque ambos cumplen la función de garantizar la redundancia y el escalado, el ReplicaSet ofrece una lógica de selección más potente.

Mientras que el Replication Controller solo admite selectores de igualdad (seleccionar Pods donde `env = production`), el ReplicaSet soporta selectores basados en conjuntos (seleccionar Pods donde `env` esté en `{production, staging}`). En la práctica moderna de Kubernetes, rara vez se gestionan los ReplicaSets de forma directa; en su lugar, el objeto Deployment los crea y administra automáticamente para orquestar las actualizaciones y el escalado de la aplicación.

Labels y Selectors

El mecanismo de Labels (etiquetas) y Selectors (selectores) constituye el núcleo de la flexibilidad organizativa de Kubernetes. Los Labels son pares clave-valor adjuntos a los objetos (como Pods) que permiten a los usuarios mapear sus propias estructuras organizativas (por ejemplo, `release: stable`, `tier: frontend`) a los recursos del sistema, sin implicar una semántica directa para el núcleo de Kubernetes.

Los Selectors, por su parte, permiten agrupar y operar sobre subconjuntos de objetos basándose en sus etiquetas. Este diseño fomenta un desacoplamiento débil entre los componentes. Por ejemplo, un Service no necesita saber las direcciones IP específicas de los Pods a los que dirige el tráfico; simplemente define un Selector para buscar cualquier Pod con una etiqueta específica (como `app: backend`). A medida que nuevos Pods con esa etiqueta se crean o destruyen, el Service actualiza dinámicamente su lista de destinos, permitiendo una arquitectura altamente dinámica y resiliente.

PODS DE SISTEMA – NAMESPACE *KUBE-SYSTEM*

En la arquitectura organizativa de Kubernetes, los namespaces funcionan como clústeres virtuales respaldados por el mismo clúster físico, diseñados para separar entornos y recursos. Entre ellos, el namespace denominado kube-system ostenta una importancia crítica, pues es el espacio reservado para los objetos creados por el sistema de Kubernetes. Este aislamiento no es meramente organizativo, sino que actúa como un mecanismo de seguridad para prevenir que los usuarios o administradores modifiquen o eliminen accidentalmente componentes esenciales para la operatividad de la infraestructura, separando claramente las cargas de trabajo de usuario de los procesos vitales del clúster.

Dentro de este espacio de nombres residen los Pods que conforman la infraestructura base y los servicios auxiliares del clúster. Dependiendo de la herramienta de despliegue utilizada (como kubectl), es común encontrar aquí los componentes del Plano de Control (Control Plane) ejecutándose como Pods estáticos o DaemonSets. Esto incluye al servidor de API (kube-apiserver), el gestor de controladores (kube-controller-manager) y el planificador (kube-scheduler), así como la base de datos etcd. La presencia de estos elementos en kube-system garantiza que el cerebro del clúster tenga prioridad y protección frente a otras aplicaciones desplegadas en espacios de nombres estándar.

Adicionalmente, kube-system aloja servicios críticos que deben ejecutarse en todos o en una selección de los nodos para garantizar la funcionalidad de red y el descubrimiento de servicios. Un ejemplo primordial es CoreDNS, el servidor DNS del clúster que permite la resolución de nombres entre Services y Pods, facilitando la comunicación interna. Asimismo, el componente kube-proxy, encargado de gestionar las reglas de red y el reenvío de tráfico en cada nodo, se despliega habitualmente en este namespace. También es el lugar donde residen los agentes de la interfaz de red (CNI), como los Pods de Calico, Flannel o Weave, asegurando que la capa de red se establezca antes de que las aplicaciones de usuario puedan operar.

REDES EN KUBERNETES

El modelo de red de Kubernetes impone una arquitectura plana en la que cada Pod posee su propia dirección IP única y en la que todos los Pods pueden comunicarse entre sí a través de los nodos sin necesidad de traducción de direcciones de red (NAT). Para implementar este modelo de manera flexible sobre diferentes infraestructuras, Kubernetes delega la configuración de la conectividad en la Interfaz de Red de Contenedores (CNI).

CNI: Calico, Flannel y Weave

Los complementos o plugins CNI son los responsables de asignar direcciones IP a los Pods y de gestionar el enrutamiento del tráfico dentro del clúster. Existen diversas implementaciones con enfoques arquitectónicos distintos, siendo Calico, Flannel y Weave algunas de las más destacadas en el ecosistema. Flannel se caracteriza por su simplicidad; crea una red superpuesta (overlay network) de capa 3 que encapsula los paquetes (habitualmente mediante VXLAN) para transportarlos entre nodos, siendo una opción ideal para configuraciones básicas que no requieren políticas de red complejas.

Por otro lado, Calico ofrece un enfoque orientado al rendimiento y la seguridad. En lugar de utilizar encapsulación, Calico emplea el protocolo de enrutamiento BGP (Border Gateway Protocol) para distribuir información de rutas entre los nodos, operando como una red pura de capa 3. Además, destaca por su soporte nativo para Network Policies, permitiendo a los administradores restringir el tráfico entre Pods por motivos de seguridad. Finalmente, Weave Net (o simplemente Weave) se centra en la facilidad de uso y la resiliencia, construyendo una red de malla (mesh) entre los nodos que puede cifrar el tráfico automáticamente, facilitando la conexión de contenedores a través de diferentes redes o nubes sin configuración adicional.

Acceso Externo: NodePort, LoadBalancer e Ingress

Si bien los objetos Service resuelven la comunicación interna, exponer las aplicaciones al tráfico externo requiere estrategias específicas. El mecanismo más básico es el NodePort, que abre un puerto específico en todas las interfaces de red de cada uno de los nodos del clúster, permitiendo el acceso al servicio mediante la combinación IP-del-Nodo:Puerto. Aunque funcional, este método presenta limitaciones en la gestión de puertos y no es ideal para entornos de producción masiva.

Para entornos alojados en proveedores de nube pública, el tipo LoadBalancer es la solución estándar. Este recurso aprovisiona automáticamente un equilibrador de carga externo nativo de la nube (como en AWS o Google Cloud) que dirige el tráfico hacia los nodos del clúster, proporcionando una dirección IP pública dedicada para el servicio. Sin embargo, para una gestión más sofisticada del tráfico HTTP y HTTPS, se utiliza el recurso Ingress. A diferencia de los anteriores, que operan en capas inferiores, el Ingress actúa como un controlador de capa 7 (aplicación) que gestiona el acceso externo basándose en reglas de enrutamiento, como nombres de dominio o rutas URL, permitiendo consolidar múltiples servicios bajo una única dirección IP pública y gestionar certificados SSL/TLS de manera centralizada.

HPA – HORIZONTAL POD AUTOSCALER

La capacidad de adaptación dinámica ante cargas de trabajo variables es una de las características más potentes de la orquestación de contenedores. En Kubernetes, el Horizontal Pod Autoscaler (HPA) es el componente encargado de automatizar el escalado horizontal de las aplicaciones, ajustando el número de réplicas de los Pods en un Deployment o ReplicaSet en función de la demanda observada. A diferencia del escalado vertical, que implica aumentar los recursos de un único nodo o contenedor, el HPA incrementa o reduce la cantidad de instancias de la aplicación, lo que favorece la distribución de la carga y la alta disponibilidad.

El funcionamiento del HPA se basa en un ciclo de control continuo que monitoriza métricas de recursos específicas, siendo el uso de CPU y memoria las más habituales. El controlador consulta periódicamente la API de métricas de recursos (o métricas personalizadas si así se configura) para comparar el consumo actual con los objetivos definidos por el usuario. Si el uso medio de los recursos supera el umbral establecido, el HPA calcula el número necesario de réplicas adicionales y actualiza el objeto de escalado correspondiente para aprovisionar nuevos Pods. Inversamente, si la carga disminuye, el sistema reduce el número de réplicas para liberar recursos del clúster, optimizando así la eficiencia operativa y los costes de infraestructura.

ALMACENAMIENTO EN KUBERNETES

La naturaleza efímera de los contenedores plantea un desafío significativo para las aplicaciones con estado (stateful): cuando un contenedor se detiene o se reinicia, los datos escritos en su sistema de archivos local se pierden. Para resolver esta problemática y garantizar la durabilidad de la información más allá del ciclo de vida de los Pods, Kubernetes implementa un subsistema de almacenamiento robusto que desacopla la infraestructura física de almacenamiento de su consumo lógico. Este sistema se articula principalmente a través de dos objetos complementarios: los Volúmenes Persistentes y las Reclamaciones de Volúmenes Persistentes.

Persistent Volumes (PV)

Un Persistent Volume (PV) es un recurso de almacenamiento provisionado en el clúster, ya sea de forma manual por un administrador o de manera dinámica mediante una Storage Class. Funciona como una abstracción de la infraestructura de almacenamiento subyacente, ocultando a los usuarios los detalles técnicos de la implementación física, ya sea un sistema de archivos en red (NFS), almacenamiento en bloque de un proveedor de nube (como AWS EBS o Google Persistent Disk) o sistemas de almacenamiento distribuido.

A diferencia de los volúmenes ordinarios que están vinculados a la vida de un Pod, el PV es un recurso del clúster con un ciclo de vida independiente. Esto significa que el objeto PV y los datos que contiene persisten incluso después de que los Pods que lo utilizaban hayan sido eliminados. El objeto PV captura los detalles de capacidad, los modos de acceso (como lectura/escritura por uno o varios nodos) y la política de reciclaje de los datos una vez liberado el recurso.

Persistent Volume Claims (PVC)

Mientras que el PV representa el recurso de almacenamiento en sí, la Persistent Volume Claim (PVC) representa la solicitud de dicho almacenamiento por parte de un usuario o aplicación. Se puede establecer una analogía directa con los recursos computacionales: de la misma manera que un Pod consume recursos de CPU y memoria de un Nodo, una PVC consume recursos de almacenamiento de un PV.

Cuando un usuario crea una PVC, especifica los requisitos de almacenamiento necesarios, como el tamaño solicitado y los modos de acceso (por ejemplo, ReadWriteOnce o ReadWriteMany). El plano de control de Kubernetes se encarga entonces de buscar un PV disponible que satisfaga estos requerimientos y los vincula (binding) de manera exclusiva. Una vez establecida esta vinculación, el Pod puede montar la PVC como un volumen, accediendo a los datos sin necesidad de conocer la infraestructura subyacente. Este mecanismo de reclamación permite a los desarrolladores desplegar aplicaciones que requieren persistencia sin acoplarse a una tecnología de almacenamiento específica, garantizando la portabilidad de la aplicación entre diferentes entornos.

CONCLUSIONES

El análisis detallado de la arquitectura y los componentes de Kubernetes permite concluir que esta plataforma se ha consolidado indiscutiblemente como el estándar de la industria para la orquestación de contenedores. Su adopción representa un cambio de paradigma en la gestión de infraestructuras de TI, evolucionando desde la administración manual de servidores hacia un modelo de administración declarativa. Este enfoque permite automatizar el ciclo de vida completo de las aplicaciones, desde el aprovisionamiento y despliegue hasta el escalado y la auto-reparación, abstrayendo la complejidad del hardware subyacente.

La robustez de Kubernetes reside en su capacidad para aprovechar primitivas eficientes del sistema operativo Linux, como los cgroups y namespaces , integrándolas en una arquitectura distribuida de tipo Maestro/Trabajador (Master/Worker) altamente escalable. A través de abstracciones lógicas como Pods, Deployments y Services, el sistema logra desacoplar las aplicaciones de la infraestructura física, garantizando la portabilidad y la alta disponibilidad. Asimismo, la flexibilidad de su modelo de red, compatible con diversos complementos CNI como Calico o Flannel , y su capacidad de gestión de almacenamiento persistente mediante Volúmenes Persistentes (PV) y Reclamaciones (PVC), demuestran que Kubernetes es apto no solo para microservicios sin estado, sino también para cargas de trabajo complejas y con estado.

En definitiva, la orquestación mediante Kubernetes ofrece una solución integral para los desafíos modernos de despliegue de software. La implementación de mecanismos avanzados como el Escalado Horizontal de Pods (HPA) asegura que los recursos se utilicen de manera óptima, adaptándose dinámicamente a la demanda. Por tanto, el dominio de esta tecnología y sus objetos fundamentales resulta imperativo para el diseño y operación de entornos de computación en la nube eficientes y resilientes.

BIBLIOGRAFÍA

- Burns, B., Beda, J., & Hightower, K. (2019). Kubernetes: Up and Running (2.^a ed.). O'Reilly Media.
- Geerling, J. (2020). Ansible for Kubernetes. Leanpub.
- Luksa, M. (2018). Kubernetes in Action. Manning Publications.
- Muschko, B. (2022). Certified Kubernetes Administrator (CKA) Study Guide. O'Reilly Media.
- Muschko, B. (2023). Certified Kubernetes Security Specialist (CKS) Study Guide. O'Reilly Media.
- Muschko, B. (2024). Certified Kubernetes Application Developer (CKAD) Study Guide. O'Reilly Media.